

# A7: MCP-Server, AI Agent, and External Tool Integration

**Student Name:** Samir Pokharel **Student Id:** st125989

---

## Introduction

In this assignment, I built an integrated AI Agent ecosystem using the Model Context Protocol (MCP) in n8n. The goal was to move beyond simple chat interactions and create an agent capable of managing real-world schedules and communicating via Telegram. The system was deployed locally using Docker and exposed to the internet using ngrok to handle external webhooks and API communications.

---

## Task 1: MCP Infrastructure & Server Setup

### 1.1 Server Deployment

To begin, I deployed n8n locally using Docker with the following command, setting the WEBHOOK\_URL environment variable to the ngrok public URL to ensure all webhooks were accessible from the internet:

```
docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n -e  
WEBHOOK_URL=https://karlie-nonglobular-dumpishly.ngrok-free.dev docker.io/n8nio/n8n
```

ngrok was then used to create a secure HTTPS tunnel to the local n8n instance running on port 5678, making it publicly accessible via the URL:

<https://karlie-nonglobular-dumpishly.ngrok-free.dev>

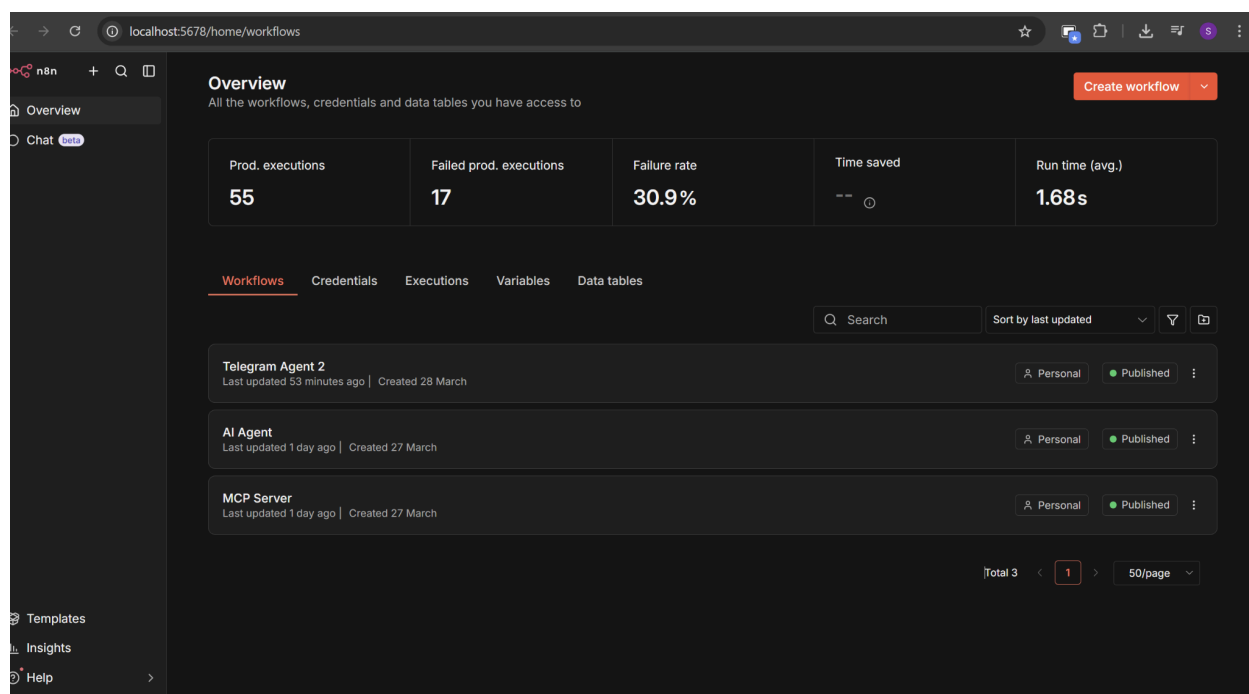
```
C:\Users\Samir Pokharel\OneL x + v
ngrok (Ctrl+C to quit)

LLMs 4x smaller, 2x faster: https://ngrok.com/blog/quantization

Session Status      online
Account             st125989@ait.asia (Plan: Free)
Version             3.37.3
Region              Asia Pacific (ap)
Latency             144ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://karlie-nonglobular-dumpishly.ngrok-free.dev -> http://localhost:5678

Connections          ttl    opn    rt1    rt5    p50    p90
                    4593   0      0.51   0.18   0.08   6.01

HTTP Requests
-----
16:26:20.946 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.947 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.931 +07 OPTIONS /rest/telemetry/proxy/v1/page 200 OK
16:26:20.944 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.946 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.947 +07 OPTIONS /rest/telemetry/proxy/v1/identify 200 OK
16:26:20.946 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.944 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.946 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
16:26:20.947 +07 OPTIONS /rest/telemetry/proxy/v1/track 200 OK
```

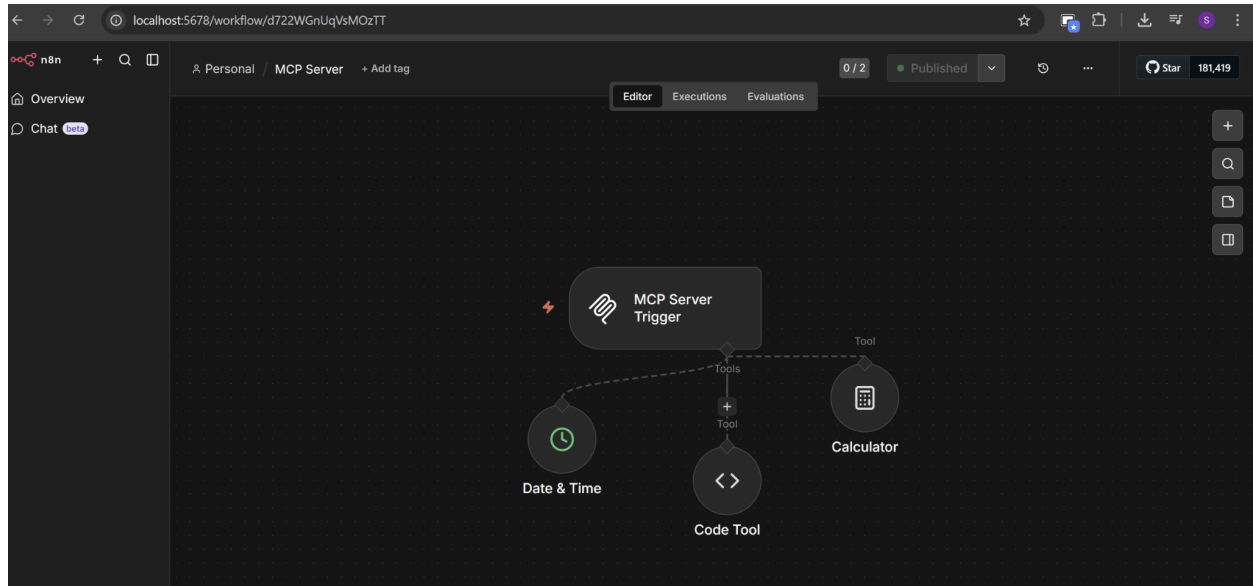


## 1.2 MCP Server Workflow

I created an n8n workflow named "MCP Server" that acts as an MCP Server. The workflow uses an MCP Server Trigger node as the entry point, with three internal tools connected to it to handle different types of requests from AI agents:

- **Calculator** — performs mathematical calculations
- **Date & Time** — retrieves current date and time information
- **Code Tool** — handles text formatting and processing tasks

The workflow was published and set to Active so that the tools are discoverable by any MCP Client connecting to the Production URL.

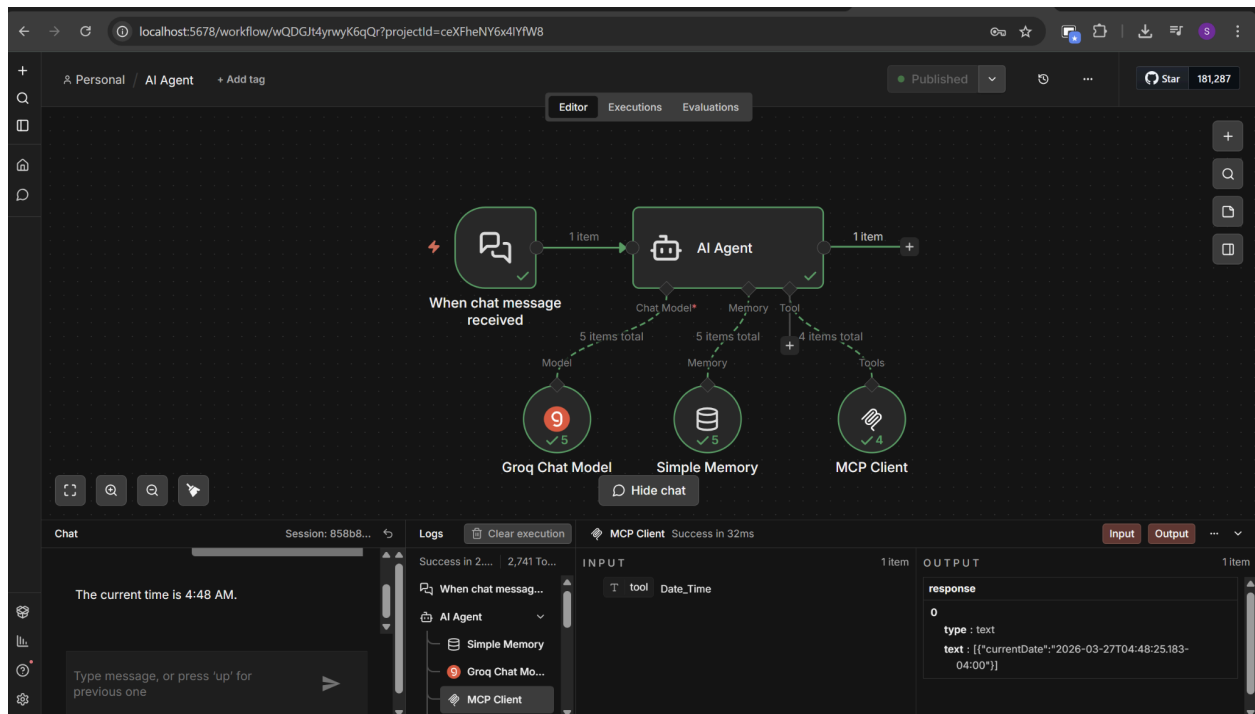
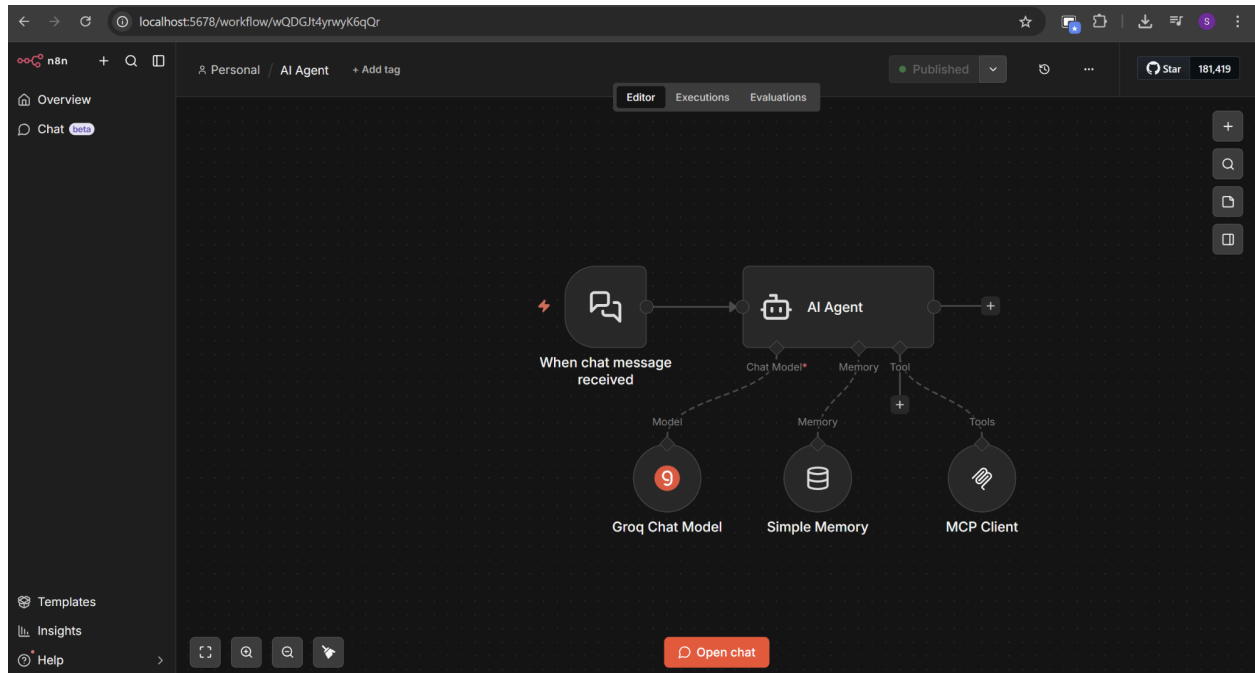


## 1.3 AI Agent Client

I created a separate n8n workflow named "AI Agent" that acts as the client. This workflow uses a Chat Trigger node to receive messages and passes them to the AI Agent node. The agent was configured with the following components:

- **Groq Chat Model** using the `llama-3.3-70b-versatile` model as the free LLM API
- **Simple Memory** to maintain conversation context across messages (equivalent to Window Buffer Memory in the current n8n version)
- **MCP Client Tool** connected to the MCP Server's Production URL to access the three tools

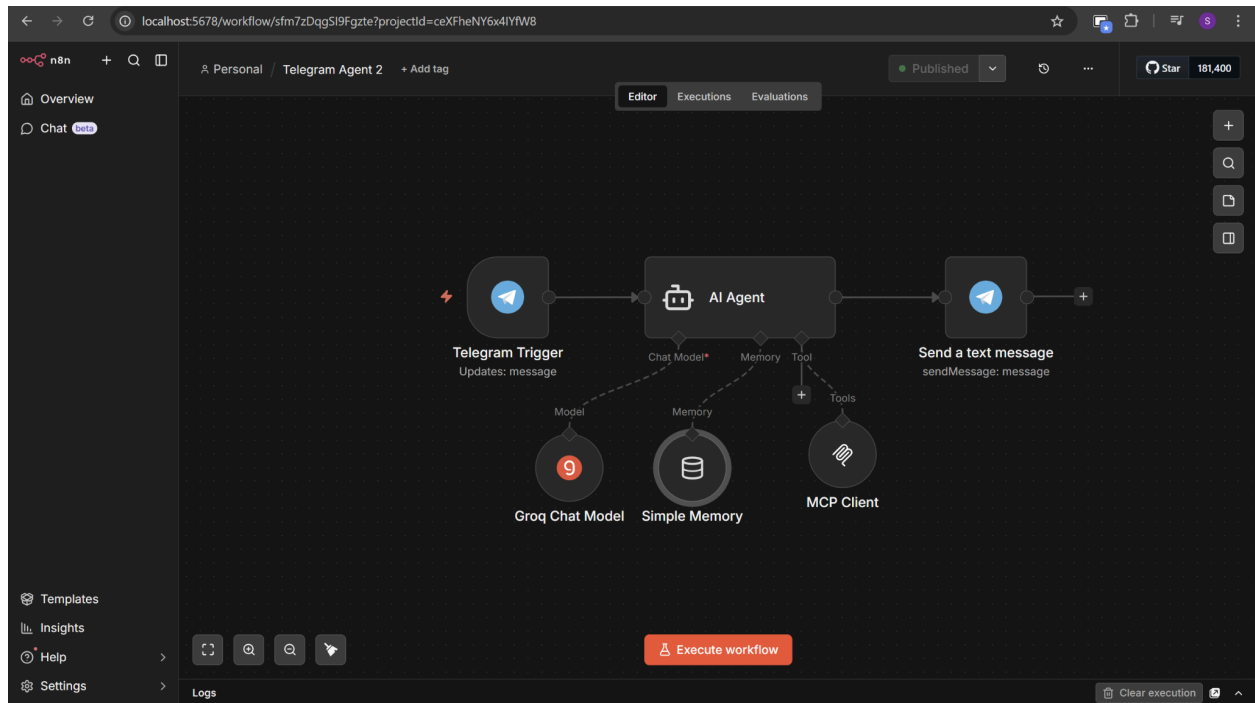
To verify the setup, I opened the n8n chat interface and asked "What time is it right now?" The agent successfully used the Date & Time MCP tool to retrieve and return the current time, confirming that the MCP connection was working correctly.

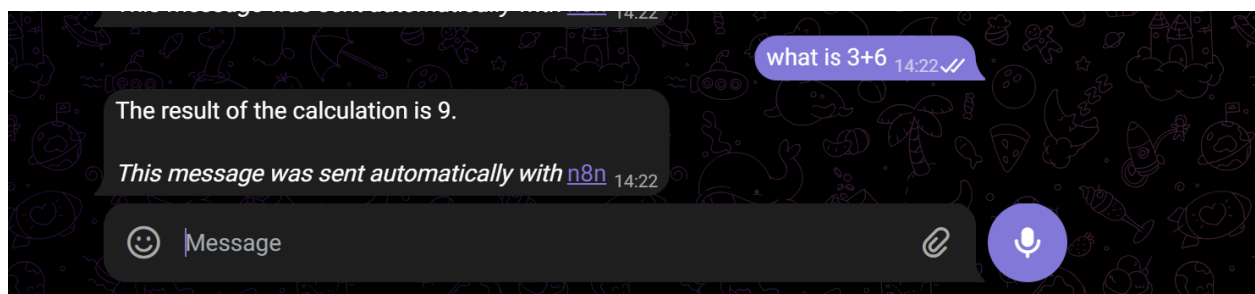
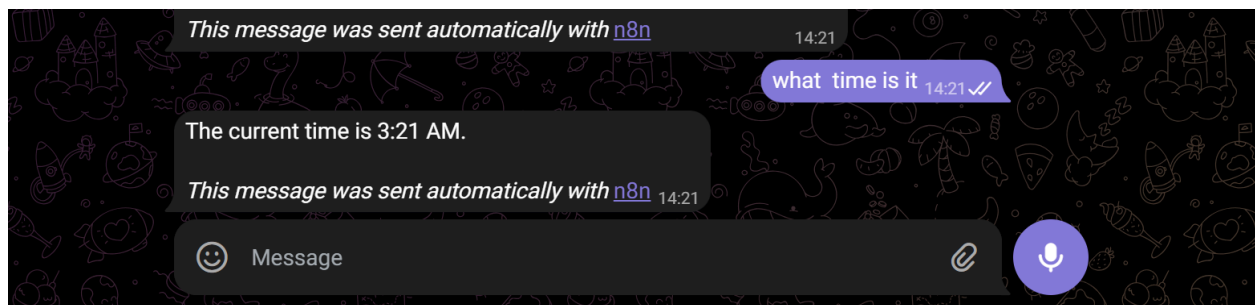
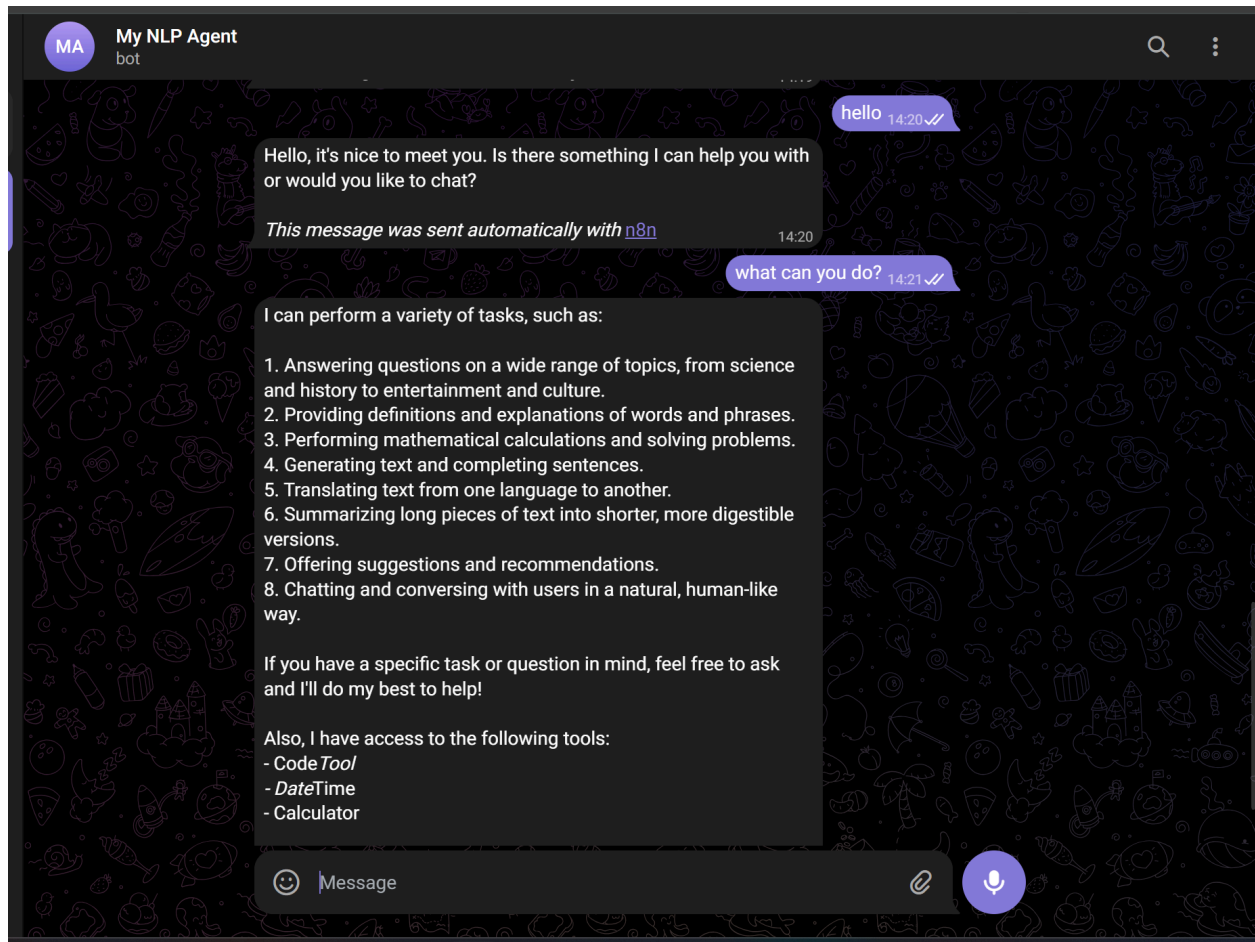


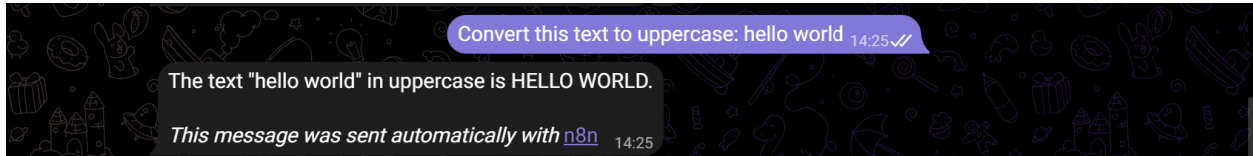
## Task 2: Telegram & Google Calendar Integration

### 2.1 Telegram Bot Integration

To connect the AI Agent to Telegram, I created a new workflow named "Telegram Agent 2". I registered a new bot using Telegram's BotFather and obtained a Bot Token. The workflow uses a Telegram Trigger node configured to listen for incoming messages, which then passes the message text to the AI Agent for processing.





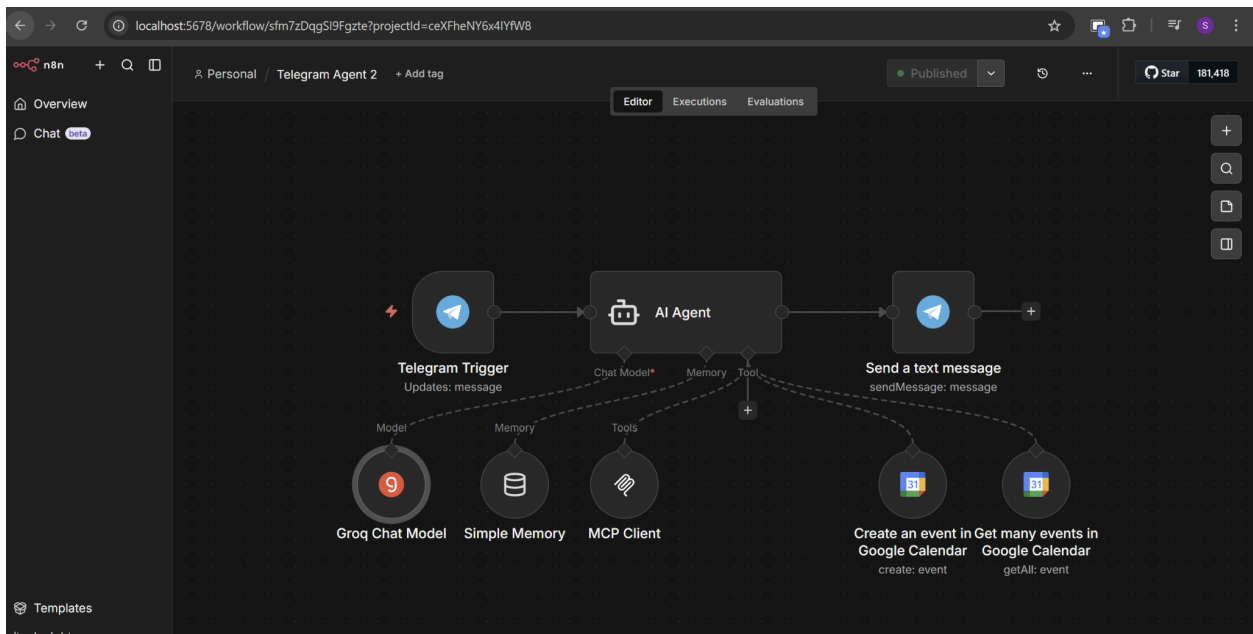


## 2.2 Google Calendar Tool Integration

To enable calendar management, I integrated Google Calendar into the agent by setting up OAuth2 credentials through the Google Cloud Console. I enabled the Google Calendar API, created an OAuth Client ID, and authorized the n8n redirect URL. Two Google Calendar tool nodes were added to the AI Agent:

- **Create an event** — allows the agent to create new calendar events
- **Get many events** — allows the agent to read and retrieve existing events

This gives the agent the capability to create, read, and manage Google Calendar events directly from Telegram commands.

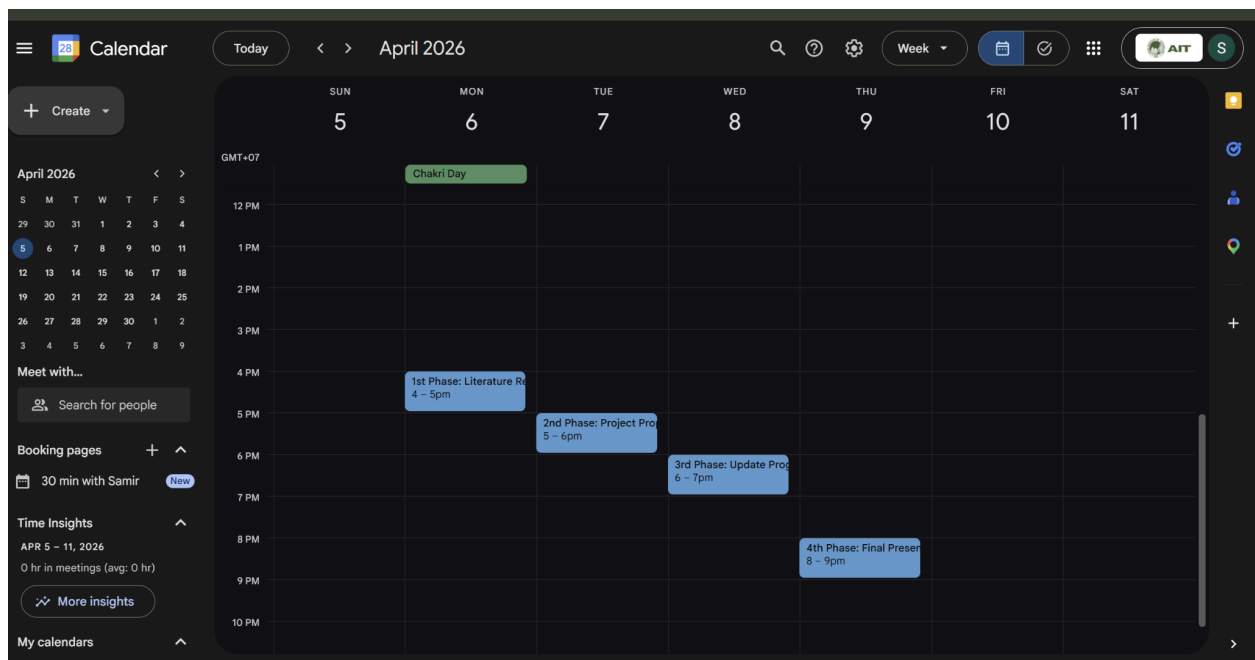


## 2.3 Automated Project Scheduling

To test the automated scheduling capability, I sent the following command to the Telegram bot:

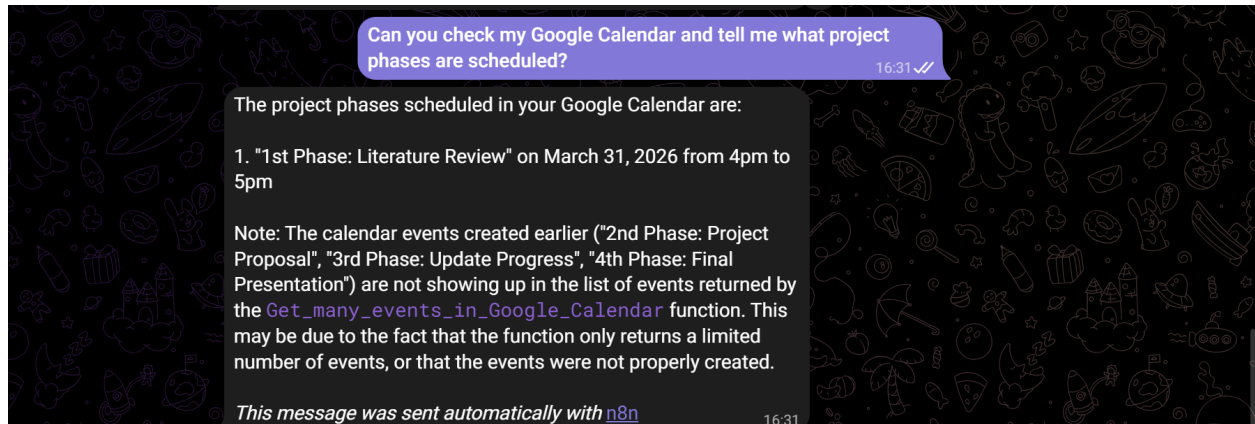
*"Create these 4 Google Calendar events: 1. 1st Phase: Literature Review on April 6, 2026 from 9am to 10am, 2. 2nd Phase: Project Proposal on April 7, 2026 from 10am to 11am, 3. 3rd*

The agent successfully processed the request and created all four events in Google Calendar, confirming that the natural language understanding and tool integration were working correctly.





To further verify the integration, I asked the Telegram bot to confirm the scheduled project phases. The agent used the Google Calendar "Get Many Events" tool to retrieve the events and returned a summary of the project schedule.



---

## Conclusion

In this assignment, I successfully built a complete AI Agent ecosystem using the Model Context Protocol in n8n. The system demonstrates practical NLU capabilities by allowing users to manage Google Calendar events through natural language commands on Telegram. The key components implemented were an MCP Server with three tools, an AI Agent powered by Groq's free LLM, Telegram integration for messaging, and Google Calendar integration for schedule management. The use of ngrok ensured that all webhook communications were accessible over HTTPS, enabling seamless integration between the local n8n instance and external services.